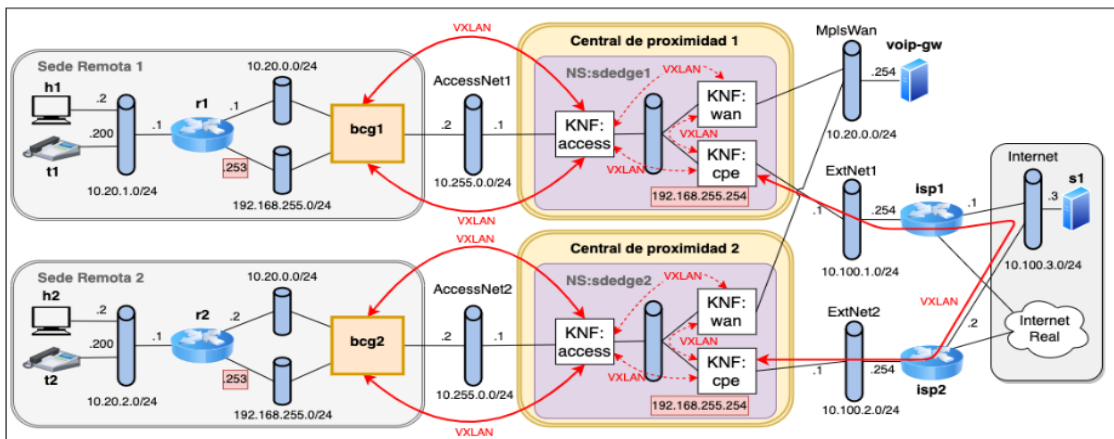


Practica 5 – Servicios SD-WAN en centrales de proximidad

Integrantes Equipo:

Karen Cardentey Pando

Preparación del Entorno y Arranque del escenario de red.



RDSV-K8S-2024-v1 [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

```
upm@vnxlab: ~/shared/sdedge-ns/vnx
upm@vnxlab: ~/shared/sdedge-ns/vnx 80x24

bcg2      | lxc          | con0: 'lxc-console -n bcg2'
          |              | con1: 'lxc-console -n bcg2'

isp1      | lxc          | con0: 'lxc-console -n isp1'
          |              | con1: 'lxc-console -n isp1'

isp2      | lxc          | con0: 'lxc-console -n isp2'
          |              | con1: 'lxc-console -n isp2'

s1        | lxc          | con0: 'lxc-console -n s1'
          |              | con1: 'lxc-console -n s1'

Total time elapsed: 40 seconds

upm@vnxlab:~/shared/sdedge-ns/vnx$
```

- h1 - console #1
- t1 - console #1
- h2 - console #1
- t2 - console #1
- r1 - console #1
- r2 - console #1
- voip-gw - console #1
- bcg1 - console #1
- bcg2 - console #1
- isp1 - console #1
- isp2 - console #1
- s1 - console #1

Pruebas de conectividad entre los nodos h1, t1 y r1, se hace ping desde h1 hacia t1(10.20.1.200) y hacia r1(10.20.1.1)

```
RDSV-K8S-2024-v1 [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

h1 - console #1
C
-- 10.20.1.1 ping statistics ---
  packets transmitted, 4 received, 0% packet loss, time 3073ms
  tt min/avg/max/mdev = 0.123/0.144/0.178/0.021 ms
oot@h1:/home/vnx# ping 10.20.1.200
PING 10.20.1.200 (10.20.1.200) 56(84) bytes of data.
 4 bytes from 10.20.1.200: icmp_seq=1 ttl=64 time=0.080 ms
 4 bytes from 10.20.1.200: icmp_seq=2 ttl=64 time=0.135 ms
 4 bytes from 10.20.1.200: icmp_seq=3 ttl=64 time=0.140 ms
 4 bytes from 10.20.1.200: icmp_seq=4 ttl=64 time=0.122 ms
 4 bytes from 10.20.1.200: icmp_seq=5 ttl=64 time=0.131 ms
 4 bytes from 10.20.1.200: icmp_seq=6 ttl=64 time=0.245 ms
 4 bytes from 10.20.1.200: icmp_seq=7 ttl=64 time=0.144 ms
C
-- 10.20.1.200 ping statistics ---
  packets transmitted, 7 received, 0% packet loss, time 6135ms
  tt min/avg/max/mdev = 0.080/0.142/0.245/0.046 ms
oot@h1:/home/vnx#
oot@h1:/home/vnx#
oot@h1:/home/vnx# ping 10.20.1.1
PING 10.20.1.1 (10.20.1.1) 56(84) bytes of data.
 4 bytes from 10.20.1.1: icmp_seq=1 ttl=64 time=0.088 ms
 4 bytes from 10.20.1.1: icmp_seq=2 ttl=64 time=0.074 ms
 4 bytes from 10.20.1.1: icmp_seq=3 ttl=64 time=0.132 ms
 4 bytes from 10.20.1.1: icmp_seq=4 ttl=64 time=0.212 ms
 4 bytes from 10.20.1.1: icmp_seq=5 ttl=64 time=0.129 ms
 4 bytes from 10.20.1.1: icmp_seq=6 ttl=64 time=0.142 ms
C
-- 10.20.1.1 ping statistics ---
  packets transmitted, 6 received, 0% packet loss, time 5115ms
  tt min/avg/max/mdev = 0.074/0.129/0.212/0.044 ms
oot@h1:/home/vnx#
```

Se hace ping desde t1 hacia h1(10.20.1.2) y hacia r1(10.20.1.1)

```
RDSV-K8S-2024-v1 [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda

t1 - console #1
vnx@t1:~$ sudo su
[sudo] password for vnx:
root@t1:/home/vnx# ping 10.20.1.2
PING 10.20.1.2 (10.20.1.2) 56(84) bytes of data.
 64 bytes from 10.20.1.2: icmp_seq=1 ttl=64 time=0.096 ms
 64 bytes from 10.20.1.2: icmp_seq=2 ttl=64 time=0.106 ms
 64 bytes from 10.20.1.2: icmp_seq=3 ttl=64 time=0.128 ms
^C
--- 10.20.1.2 ping statistics ---
 3 packets transmitted, 3 received, 0% packet loss, time 2052ms
 rtt min/avg/max/mdev = 0.096/0.110/0.128/0.013 ms
root@t1:/home/vnx# 10.20.1.1
bash: 10.20.1.1: command not found
root@t1:/home/vnx# ping 10.20.1.1
PING 10.20.1.1 (10.20.1.1) 56(84) bytes of data.
 64 bytes from 10.20.1.1: icmp_seq=1 ttl=64 time=0.206 ms
 64 bytes from 10.20.1.1: icmp_seq=2 ttl=64 time=0.102 ms
 64 bytes from 10.20.1.1: icmp_seq=3 ttl=64 time=0.251 ms
^C
--- 10.20.1.1 ping statistics ---
 3 packets transmitted, 3 received, 0% packet loss, time 2035ms
 rtt min/avg/max/mdev = 0.102/0.186/0.251/0.062 ms
root@t1:/home/vnx#
```

Se hace ping desde r1 hacia h1(10.20.1.2) y hacia t1(10.20.1.200)

```
r1 - console #1
root@r1:/home/vnx#
root@r1:/home/vnx#
root@r1:/home/vnx# ping 10.20.1.2
PING 10.20.1.2 (10.20.1.2) 56(84) bytes of data.
 64 bytes from 10.20.1.2: icmp_seq=1 ttl=64 time=0.092 ms
 64 bytes from 10.20.1.2: icmp_seq=2 ttl=64 time=0.097 ms
 64 bytes from 10.20.1.2: icmp_seq=3 ttl=64 time=0.099 ms
 64 bytes from 10.20.1.2: icmp_seq=4 ttl=64 time=0.090 ms
 64 bytes from 10.20.1.2: icmp_seq=5 ttl=64 time=0.131 ms

--- 10.20.1.2 ping statistics ---
 5 packets transmitted, 5 received, 0% packet loss, time 4089ms
 rtt min/avg/max/mdev = 0.090/0.101/0.131/0.014 ms
^Croot@r1:/home/vnx#
root@r1:/home/vnx#
root@r1:/home/vnx#
root@r1:/home/vnx# ping 10.20.1.200
PING 10.20.1.200 (10.20.1.200) 56(84) bytes of data.
 64 bytes from 10.20.1.200: icmp_seq=1 ttl=64 time=0.094 ms
 64 bytes from 10.20.1.200: icmp_seq=2 ttl=64 time=0.177 ms
 64 bytes from 10.20.1.200: icmp_seq=3 ttl=64 time=0.133 ms
 64 bytes from 10.20.1.200: icmp_seq=4 ttl=64 time=0.132 ms
 64 bytes from 10.20.1.200: icmp_seq=5 ttl=64 time=0.055 ms
^C
--- 10.20.1.200 ping statistics ---
 5 packets transmitted, 5 received, 0% packet loss, time 4104ms
 rtt min/avg/max/mdev = 0.055/0.118/0.177/0.041 ms
root@r1:/home/vnx#
```

Pruebas de conectividad en el segmento de Internet (10.100.3.0/24), se prueba la conectividad entre isp1, isp2 y s1:

Se hace ping desde isp1 hacia isp2(10.100.3.2) y hacia s1(10.100.3.3)

```
isp1 - console #1
Last login: Mon Oct 11 15:00:50 UTC 2021 on console
vnx@isp1:~$ sudo su
[sudo] password for vnx:
root@isp1:/home/vnx# ping 10.100.3.2
PING 10.100.3.2 (10.100.3.2) 56(84) bytes of data:
64 bytes from 10.100.3.2: icmp_seq=1 ttl=64 time=0.671 ms
64 bytes from 10.100.3.2: icmp_seq=2 ttl=64 time=0.101 ms
64 bytes from 10.100.3.2: icmp_seq=3 ttl=64 time=0.044 ms
^C
--- 10.100.3.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2042ms
rtt min/avg/max/mdev = 0.044/0.272/0.671/0.283 ms
root@isp1:/home/vnx# ping 10.100.3.3
PING 10.100.3.3 (10.100.3.3) 56(84) bytes of data:
64 bytes from 10.100.3.3: icmp_seq=1 ttl=64 time=0.625 ms
64 bytes from 10.100.3.3: icmp_seq=2 ttl=64 time=0.086 ms
64 bytes from 10.100.3.3: icmp_seq=3 ttl=64 time=0.125 ms
^C
--- 10.100.3.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2037ms
rtt min/avg/max/mdev = 0.086/0.278/0.625/0.245 ms
root@isp1:/home/vnx#
```

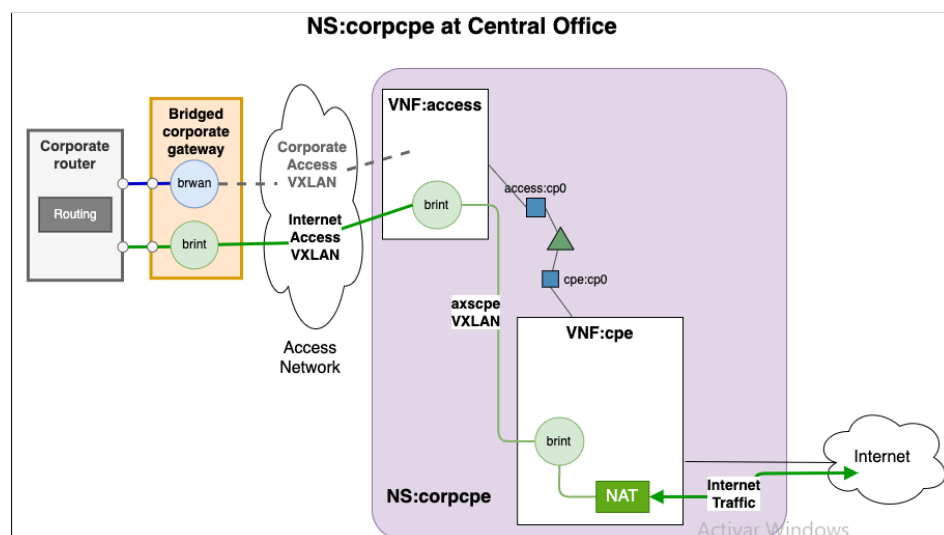
Se hace ping desde isp2 hacia isp1(10.100.3.1) y hacia s1(10.100.3.3)

```
isp2 - console #1
root@isp2:/home/vnx# ping 10.100.3.1
PING 10.100.3.1 (10.100.3.1) 56(84) bytes of data:
64 bytes from 10.100.3.1: icmp_seq=1 ttl=64 time=0.362 ms
64 bytes from 10.100.3.1: icmp_seq=2 ttl=64 time=0.095 ms
64 bytes from 10.100.3.1: icmp_seq=3 ttl=64 time=0.102 ms
64 bytes from 10.100.3.1: icmp_seq=4 ttl=64 time=0.086 ms
^C
--- 10.100.3.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3066ms
rtt min/avg/max/mdev = 0.086/0.161/0.362/0.116 ms
root@isp2:/home/vnx# ping 10.100.3.3
PING 10.100.3.3 (10.100.3.3) 56(84) bytes of data:
64 bytes from 10.100.3.3: icmp_seq=1 ttl=64 time=0.756 ms
64 bytes from 10.100.3.3: icmp_seq=2 ttl=64 time=0.150 ms
64 bytes from 10.100.3.3: icmp_seq=3 ttl=64 time=0.101 ms
^C
--- 10.100.3.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2028ms
rtt min/avg/max/mdev = 0.101/0.335/0.756/0.297 ms
root@isp2:/home/vnx#
```

Se hace ping desde r1 hacia isp1(10.100.3.1) y hacia isp2(10.100.3.2). También se comprueba el acceso a internet desde s1.

```
s1 - console #1
[sudo] password for vnx:
root@s1:/home/vnx# ping 10.100.3.1
PING 10.100.3.1 (10.100.3.1) 56(84) bytes of data:
64 bytes from 10.100.3.1: icmp_seq=1 ttl=64 time=0.420 ms
64 bytes from 10.100.3.1: icmp_seq=2 ttl=64 time=0.098 ms
64 bytes from 10.100.3.1: icmp_seq=3 ttl=64 time=0.087 ms
^C
--- 10.100.3.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2049ms
rtt min/avg/max/mdev = 0.087/0.201/0.420/0.154 ms
root@s1:/home/vnx# ping 10.100.3.2
PING 10.100.3.2 (10.100.3.2) 56(84) bytes of data:
64 bytes from 10.100.3.2: icmp_seq=1 ttl=64 time=0.263 ms
64 bytes from 10.100.3.2: icmp_seq=2 ttl=64 time=0.079 ms
64 bytes from 10.100.3.2: icmp_seq=3 ttl=64 time=0.104 ms
^C
--- 10.100.3.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2025ms
rtt min/avg/max/mdev = 0.079/0.148/0.263/0.081 ms
root@s1:/home/vnx# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=253 time=10.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=253 time=15.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=253 time=10.7 ms
^C
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2015ms
rtt min/avg/max/mdev = 10.418/12.283/15.712/2.427 ms
root@s1:/home/vnx#
```

4. Servicio de red corpce



4.1 (P) Imágenes vnf-access y vnf-cpe

Identifique el nombre de la imagen estándar de partida para la creación de vnf-access y de vnf-cpe (líneas FROM), y qué paquetes adicionales se están instalando en cada caso.

```
Dockerfile
1 FROM ubuntu:20.04
2 # install required packages
3 RUN apt-get clean
4 RUN apt-get update \
5     && apt-get install -y \
6     net-tools \
7     traceroute \
8     build-essential \
9     inetutils-ping \
10    bridge-utils \
11    tcpdump \
12    openvswitch-switch \
13    openvswitch-common \
14    iperf \
15    iproute2 \
16    curl \
17    nano \
18    vim
19
20
```

```
Dockerfile
1 FROM educaredes/vnf-access
2 # install required packages
3 RUN apt-get clean
4 RUN apt-get update \
5     && apt-get install -y \
6     iptables
7
8 COPY vnx_config_nat vnx_config_nat
9
10 RUN chmod +x vnx_config_nat
11
```

El nombre de la imagen estándar de partida para la creación de vnf-access sería: ubuntu:20.04, y los paquetes adicionales instalados son: net-tools, traceroute, build-essential, inetutils-ping, bridge-utils, tcpdump, openvswitch-switch, openvswitch-common, iperf, iproute2, curl, nano y vim.

El nombre de la imagen estándar de partida para la creación de vnf-cpe sería: educaredes/vnf-access, y el paquete adicional instalado es: iptables.

4.2 Instanciación de corpce1

Instancia del servicio que dará acceso a Internet a la sede 1

```
cd ~/shared/sdedge-ns
./cpe1.sh
```

```

upm@vnxlab: ~/shared/sdedge-ns
upm@vnxlab: ~/shared/sdedge-ns 150x32

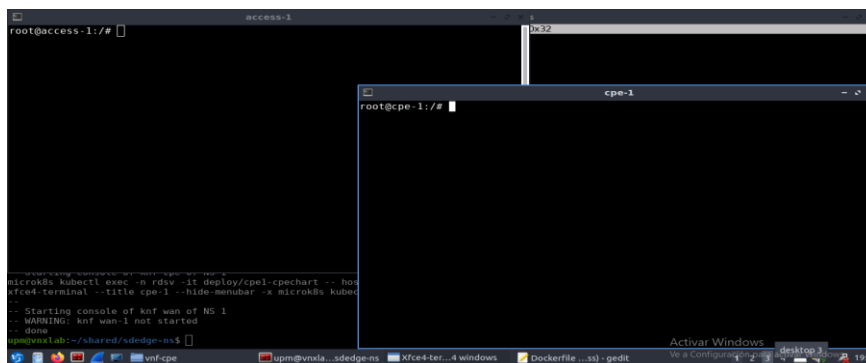
...
## 1. Obtener IPs de las VNFs
IPACCESS = 10.1.138.6
IPCPE = 10.1.138.7
## 2. Iniciar el Servicio OpenVirtualSwitch en cada VNF
* /etc/openvswitch/conf.db does not exist
* Creating empty database /etc/openvswitch/conf.db
* Starting ovsdb-server
* Configuring Open vSwitch system IDs
* Starting ovs-vswitchd
* Enabling remote OVSDB managers
* /etc/openvswitch/conf.db does not exist
* Creating empty database /etc/openvswitch/conf.db
* Starting ovsdb-server
* Configuring Open vSwitch system IDs
* Starting ovs-vswitchd
* Enabling remote OVSDB managers
## 3. En VNF:access agregar un bridge y configurar IPs y rutas
## 4. En VNF:cpe agregar un bridge y configurar IPs y rutas
## 5. En VNF:cpe activar NAT para dar salida a Internet
Adding NAT rules (int_if=brint, ext_if=net1)
Adding rule #1...
MASQUERADE all opt -- in * out net1 0.0.0.0/0 -> 0.0.0.0/0
Adding rule #2...
ACCEPT all opt -- in net1 out brint 0.0.0.0/0 -> 0.0.0.0/0 state RELATED,ESTABLISHED
Adding rule #3...
ACCEPT all opt -- in brint out net1 0.0.0.0/0 -> 0.0.0.0/0
-
K8s deployments para la red 1:
deploy/access1-accesschart
deploy/cpe1-cpechart
upm@vnxlab:~/shared/sdedge-ns$

```

Acceder a los terminales de las KNFs usando el comando:

`cd ~/shared/sdedge-ns`

`bin/sdw-knf-consoles open 1`



Para las pruebas de conectividad desde la KNF de access-1 buscamos la dirección ip de eth0 con el comando `ip a`, y hacemos ping al 10.1.138.6 desde cpe-1.

access-1	cpe-1
<pre> root@access-1:/# ip a 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid lft forever preferred_lft forever inet6 ::1/128 scope host valid lft forever preferred_lft forever 3: eth0@if57: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue st link/ether 92:5e:09:28:43:7c brd ff:ff:ff:ff:ff:ff link-netnsid 0 inet 10.1.138.6/32 scope global eth0 valid lft forever preferred_lft forever inet6 fe80::905e:9ff:fe28:437c/64 scope link valid lft forever preferred_lft forever 4: net1@if60: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue st link/ether be:35:bb:98:8f:1a brd ff:ff:ff:ff:ff:ff link-netnsid 0 inet 10.255.0.1/24 brd 10.255.0.255 scope global net1 valid lft forever preferred_lft forever inet6 fe80::bc35:bbff:fe98:8f1a/64 scope link valid lft forever preferred_lft forever </pre>	<pre> root@cpe-1:/# ping 10.1.138.6 PING 10.1.138.6 (10.1.138.6): 56 data bytes 64 bytes from 10.1.138.6: icmp_seq=0 ttl=63 time=0.169 ms 64 bytes from 10.1.138.6: icmp_seq=1 ttl=63 time=0.073 ms 64 bytes from 10.1.138.6: icmp_seq=2 ttl=63 time=0.164 ms 64 bytes from 10.1.138.6: icmp_seq=3 ttl=63 time=0.150 ms 64 bytes from 10.1.138.6: icmp_seq=4 ttl=63 time=0.157 ms ^C--- 10.1.138.6 ping statistics --- 5 packets transmitted, 5 packets received, 0% packet loss round-trip min/avg/max/stddev = 0.073/0.143/0.169/0.035 ms root@cpe-1:/# </pre>

Hacemos el proceso inverso, y damos ping desde la KNF de access-1 hacia cpe-1 donde la ip de eth0 es la 10.1.138.7.

cpe-1	access-1
<pre> 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid lft forever preferred_lft forever inet6 ::1/128 scope host valid lft forever preferred_lft forever 3: eth0@if58: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue st group default link/ether de:a7:63:c9:60:75 brd ff:ff:ff:ff:ff:ff inet 10.1.138.7/32 scope global eth0 valid lft forever preferred_lft forever inet6 fe80::dea7:63ff:ec9:6075/64 scope link valid lft forever preferred_lft forever 4: net1@if59: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue st group default link/ether 5e:43:f2:24:de:81 brd ff:ff:ff:ff:ff:ff inet 10.100.1.1/24 brd 10.100.1.255 scope global net1 valid lft forever preferred_lft forever inet6 fe80::5e43:f2ff:fe24:de81/64 scope link valid lft forever preferred_lft forever 5: net2@if61: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue st group default link/ether d2:dc:1b:74:aa:29 brd ff:ff:ff:ff:ff:ff </pre>	<pre> root@access-1:/# root@access-1:/# root@access-1:/# ping 10.1.138.7 PING 10.1.138.7 (10.1.138.7): 56 data bytes 64 bytes from 10.1.138.7: icmp_seq=0 ttl=63 time=0.157 ms 64 bytes from 10.1.138.7: icmp_seq=1 ttl=63 time=0.127 ms 64 bytes from 10.1.138.7: icmp_seq=2 ttl=63 time=0.181 ms 64 bytes from 10.1.138.7: icmp_seq=3 ttl=63 time=0.154 ms 64 bytes from 10.1.138.7: icmp_seq=4 ttl=63 time=0.155 ms 64 bytes from 10.1.138.7: icmp_seq=5 ttl=63 time=0.083 ms 64 bytes from 10.1.138.7: icmp_seq=6 ttl=63 time=0.087 ms 64 bytes from 10.1.138.7: icmp_seq=7 ttl=63 time=0.326 ms 64 bytes from 10.1.138.7: icmp_seq=8 ttl=63 time=0.170 ms 64 bytes from 10.1.138.7: icmp_seq=9 ttl=63 time=0.235 ms 64 bytes from 10.1.138.7: icmp_seq=10 ttl=63 time=0.278 ms 64 bytes from 10.1.138.7: icmp_seq=11 ttl=63 time=0.067 ms 64 bytes from 10.1.138.7: icmp_seq=12 ttl=63 time=0.142 ms 64 bytes from 10.1.138.7: icmp_seq=13 ttl=63 time=0.284 ms 64 bytes from 10.1.138.7: icmp_seq=14 ttl=63 time=0.090 ms 64 bytes from 10.1.138.7: icmp_seq=15 ttl=63 time=0.076 ms 64 bytes from 10.1.138.7: icmp_seq=16 ttl=63 time=0.062 ms 64 bytes from 10.1.138.7: icmp_seq=17 ttl=63 time=0.165 ms 64 bytes from 10.1.138.7: icmp_seq=18 ttl=63 time=0.110 ms 64 bytes from 10.1.138.7: icmp_seq=19 ttl=63 time=0.235 ms </pre>

4.3 (P) Análisis de las conexiones a redes externas y configuración

A partir del contenido del script `cpe1.sh` y los demás scripts que se llaman desde este, analice y describa resumidamente los pasos que se están siguiendo para realizar la conexión a las redes externas y la configuración del servicio.

Vamos a analizar los 3 scripts individualmente para entender los pasos que se siguen.

Visualizamos el script `cpe1.sh`

```
done
upm@vnxlab:~/shared/sdedge-ns$ cat cpe1.sh
#!/bin/bash
export SDWNS # needs to be defined in calling shell
export SIID="$NSID1" # $NSID1 needs to be defined in calling shell

export NETNUM=1 # used to select external networks

# CUSTUNIP: the ip address for the home side of the tunnel
export CUSTUNIP="10.255.0.2"

# CUSTPREFIX: the customer private prefix
export CUSTPREFIX="10.20.1.0/24"

# VNFTUNIP: the ip address for the vnf side of the tunnel
export VNFTUNIP="10.255.0.1"

# VCPEPUBIP: the public ip address for the vcpe
export VCPEPUBIP="10.100.1.1"

# VCPEGW: the default gateway for the vcpe
export VCPEGW="10.100.1.254"

# OSM SECTION
#./osm_corpcpe_start.sh

# HELM SECTION
./k8s_corpcpe_start.shupm@vnxlab:~/shared/sdedge-ns$
upm@vnxlab:~/shared/sdedge-ns$
```

El script `cpe1.sh` está actuando como un controlador inicial, en él se declaran las variables que se van a utilizar en la red, las necesarias para la configuración, como direcciones IP de los túneles, también las direcciones privadas y públicas, y se delega la configuración al próximo script, hace la función general de orquestación. Finalmente llama al script `k8s_corpcpe_start.sh` para realizar la configuración en Kubernetes.

Visualizamos el script `k8s_corpcpe_start.sh`

```
./k8s_corpcpe_start.shupm@vnxlab:~/shared/sdedge-ns$
upm@vnxlab:~/shared/sdedge-ns$ cat k8s_corpcpe_start.sh
#!/bin/bash

# Requires the following variables
# SDWNS: namespace in the cluster vim
# NETNUM: used to select external networks
# CUSTUNIP: the ip address for the customer side of the tunnel
# VNFTUNIP: the ip address for the vnf side of the tunnel
# VCPEPUBIP: the public ip address for the vcpe
# VCPEGW: the default gateway for the vcpe

set -u # to verify variables are defined
: $SDWNS
: $NETNUM
: $CUSTUNIP
: $VNFTUNIP
: $VCPEPUBIP
: $VCPEGW

export KUBECTL="microk8s kubectl"

## 0. Instalación
echo "## 0. Instalación de las vnfs"

echo "### 0.1 Limpieza (ignorar errores)"
for vnf in access cpe
do
    helm -n $SDWNS uninstall $vnf$NETNUM
done

echo "### 0.2 Creación de contenedores"

chart_suffix="chart-0.1.0.tgz"
for vnf in access cpe
do
    echo "### $vnf$NETNUM"
    helm -n $SDWNS install $vnf$NETNUM $vnf$chart_suffix
done

for i in {1..30}; do echo -n "."; sleep 1; done
echo ""

export VACC="deploy/access$NETNUM-accesschart"
export VCPE="deploy/cpe$NETNUM-cpechart"

./start_corpcpe.sh

echo "..."
echo "K8s deployments para la red $NETNUM:"
echo $VACC
echo $VCPE
upm@vnxlab:~/shared/sdedge-ns$
```

En el script `k8s_corpcpe_start.sh`, se hace la configuración en Kubernetes, se obtienen las IDs de despliegues de las VNFs para poder acceder a los contenedores.

Se comienza con una limpieza inicial desinstalando cualquier configuración previa de las VNFs. Después se realiza el despliegue de los contenedores, con Helm se instalan los gráficos (charts) de access y cpe en el

clúster de Kubernetes, posteriormente se realiza una pausa asegurando que los contenedores se inicialicen bien. Se exportan las variables y se establecen los identificadores (deploy/access, deploy/cpe) que hacen referencia a los despliegues creados. Finalmente llama al script start_corpcpe.sh para configurar las funciones específicas dentro de los contenedores.

Visualizamos el script start_corpcpe.sh

```
upm@vnxlab: ~/shared/sdedge-ns$ cat start_corpcpe.sh
#!/bin/bash

# Requires the following variables
# KUBECTL: kubectl command
# SDWNS: cluster namespace in the cluster vim
# NETNUM: used to select external networks
# VACC: "pod_id" or "deployment_id" of the access vnf
# VCPE: "pod_id" or "deployment_id" of the cpd vnf
# CUSTUNIP: the ip address for the customer side of the tunnel
# VNFTUNIP: the ip address for the vnf side of the tunnel
# VCPEPUBIP: the public ip address for the vcpe
# VCPEGW: the default gateway for the vcpe

set -u # to verify variables are defined
: $KUBECTL
: $SDWNS
: $NETNUM
: $VACC
: $VCPE
: $CUSTUNIP
: $CUSTPREFIX
: $VNFTUNIP
: $VCPEPUBIP
: $VCPEGW

if [[ ! $VACC =~ "-accesschart" ]]; then
    echo ""
    echo "ERROR: incorrect <access_deployment_id>: $VACC"
    exit 1
fi

if [[ ! $VCPE =~ "-cpechart" ]]; then
    echo ""
    echo "ERROR: incorrect <cpe_deployment_id>: $VCPE"
    exit 1
fi

ACC_EXEC="$KUBECTL exec -n $SDWNS $VACC --"
CPE_EXEC="$KUBECTL exec -n $SDWNS $VCPE --"

# IP privada por defecto para el VCPE
VCPEPRIVIP="192.168.255.254"
# IP privada por defecto para el router del cliente
CUSTGW="192.168.255.253"

# Router por defecto inicial en k8s (calico)
K8SGW="169.254.1.1"

## 1. Obtener IPs de las VNFs
echo "## 1. Obtener IPs de las VNFs"
IPACCESS=`$ACC_EXEC hostname -I | awk '{print $1}'`
echo "IPACCESS = $IPACCESS"

IPCPE=`$CPE_EXEC hostname -I | awk '{print $1}'`
echo "IPCPE = $IPCPE"
```

```
upm@vnxlab: ~/shared/sdedge-ns 150$ cat start_corpcpe.sh
## 2. Iniciar el Servicio OpenVirtualSwitch en cada VNF:
echo "## 2. Iniciar el Servicio OpenVirtualSwitch en cada VNF"
$ACC_EXEC service openvswitch-switch start
$CPE_EXEC service openvswitch-switch start

## 3. En VNF:access agregar un bridge y configurar IPs y rutas
echo "## 3. En VNF:access agregar un bridge y configurar IPs y rutas"
$ACC_EXEC ovs-vsctl add-br brint
$ACC_EXEC ifconfig net$NETNUM $VNFTUNIP/24
$ACC_EXEC ip link add vxlan2 type vxlan id 2 remote $CUSTUNIP dstport 8742 dev net$NETNUM
$ACC_EXEC ip link add axscpe type vxlan id 4 remote $IPCPE dstport 8742 dev eth0
$ACC_EXEC ovs-vsctl add-port brint vxlan2
$ACC_EXEC ovs-vsctl add-port brint axscpe
$ACC_EXEC ifconfig vxlan2 up
$ACC_EXEC ifconfig axscpe up

## 4. En VNF:cpe agregar un bridge y configurar IPs y rutas
echo "## 4. En VNF:cpe agregar un bridge y configurar IPs y rutas"
$CPE_EXEC ovs-vsctl add-br brint
$CPE_EXEC ifconfig brint $VCPEPRIVIP/24
$CPE_EXEC ip link add axscpe type vxlan id 4 remote $IPACCESS dstport 8742 dev eth0
$CPE_EXEC ovs-vsctl add-port brint axscpe
$CPE_EXEC ifconfig axscpe up
$CPE_EXEC ifconfig brint mtu 1400
$CPE_EXEC ifconfig net$NETNUM $VCPEPUBIP/24
$CPE_EXEC ip route add $IPACCESS/32 via $K8SGW
$CPE_EXEC ip route del 0.0.0.0/0 via $K8SGW
$CPE_EXEC ip route add 0.0.0.0/0 via $VCPEGW
$CPE_EXEC ip route add $CUSTPREFIX via $CUSTGW

## 5. En VNF:cpe activar NAT para dar salida a Internet
echo "## 5. En VNF:cpe activar NAT para dar salida a Internet"
```

En el script start_corpcpe.sh va a tener la configuración interna de los VNFs

Se realizan las validaciones iniciales, comprobando que las variables estén definidas y los nombres de despliegue sean correctos, si no lo son se termina el proceso, como se muestra en la captura siguiente:

```
if [[ ! $VACC =~ "-accesschart" ]]; then
    echo ""
    echo "ERROR: incorrect <access_deployment_id>: $VACC"
    exit 1
fi

if [[ ! $VCPE =~ "-cpechart" ]]; then
    echo ""
    echo "ERROR: incorrect <cpe_deployment_id>: $VCPE"
    exit 1
fi
```

Después se obtienen las direcciones IP de los VNFs Access y cpe:

```
## 1. Obtener IPs de las VNFs
echo "## 1. Obtener IPs de las VNFs"
IPACCESS=`$ACC_EXEC hostname -I | awk '{print $1}'`
echo "IPACCESS = $IPACCESS"

IPCPE=`$CPE_EXEC hostname -I | awk '{print $1}'`
echo "IPCPE = $IPCPE"
```

Posteriormente se inicializan los servicios de Open Virtual Switch (OVS) en ambos VNFs:

```
## 2. Iniciar el Servicio OpenVirtualSwitch en cada VNF:
echo "## 2. Iniciar el Servicio OpenVirtualSwitch en cada VNF"
$ACC_EXEC service openvswitch-switch start
$CPE_EXEC service openvswitch-switch start
```

Tras haber inicializados Open Virtual Switch (OVS) se van a configurar las VNF, comenzando con VNF: access, se crea un bridge (brint) y se añaden túneles VXLAN para conectar redes.

vxlan2: Túnel hacia el lado del cliente.

axscpe: Túnel hacia el VNF cpe.

Una vez que ha configurado las interfaces, las levanta.

```
## 3. En VNF:access agregar un bridge y configurar IPs y rutas
echo "## 3. En VNF:access agregar un bridge y configurar IPs y rutas"
$ACC_EXEC ovs-vsctl add-br brint
$ACC_EXEC ifconfig net$NETNUM $VNFUNIP/24
$ACC_EXEC ip link add vxlan2 type vxlan id 2 remote $CUSTUNIP dstport 8742 dev net$NETNUM
$ACC_EXEC ip link add axscpe type vxlan id 4 remote $IPCPE dstport 8742 dev eth0
$ACC_EXEC ovs-vsctl add-port brint vxlan2
$ACC_EXEC ovs-vsctl add-port brint axscpe
$ACC_EXEC ifconfig vxlan2 up
$ACC_EXEC ifconfig axscpe up
```

Tras haber terminado con VNF:access, se procede a configurar VNF:cpe. Crea un bridge (brint) y establece la IP privada para el cliente (192.168.255.254), y también las rutas de red para alcanzar redes de cliente y acceso a Internet.

```
## 4. En VNF:cpe agregar un bridge y configurar IPs y rutas
echo "## 4. En VNF:cpe agregar un bridge y configurar IPs y rutas"
$CPE_EXEC ovs-vsctl add-br brint
$CPE_EXEC ifconfig brint $VCPEPRIVIP/24
$CPE_EXEC ip link add axscpe type vxlan id 4 remote $IPACCESS dstport 8742 dev eth0
$CPE_EXEC ovs-vsctl add-port brint axscpe
$CPE_EXEC ifconfig axscpe up
$CPE_EXEC ifconfig brint mtu 1400
$CPE_EXEC ifconfig net$NETNUM $VCPEPUBIP/24
$CPE_EXEC ip route add $IPACCESS/32 via $K8SGW
$CPE_EXEC ip route del 0.0.0.0/0 via $K8SGW
$CPE_EXEC ip route add 0.0.0.0/0 via $VCPEGW
$CPE_EXEC ip route add $CUSTPREFIX via $CUSTGW
```

En las últimas líneas se activa la NAT en el VNF:cpe para dar salida a Internet, utilizando el script /vnx_config_nat

```
## 5. En VNF:cpe activar NAT para dar salida a Internet
echo "## 5. En VNF:cpe activar NAT para dar salida a Internet"
$CPE_EXEC /vnx_config_nat brint net$NETNUM upm@vnxlab:~/shared/sdedge-ns$
```

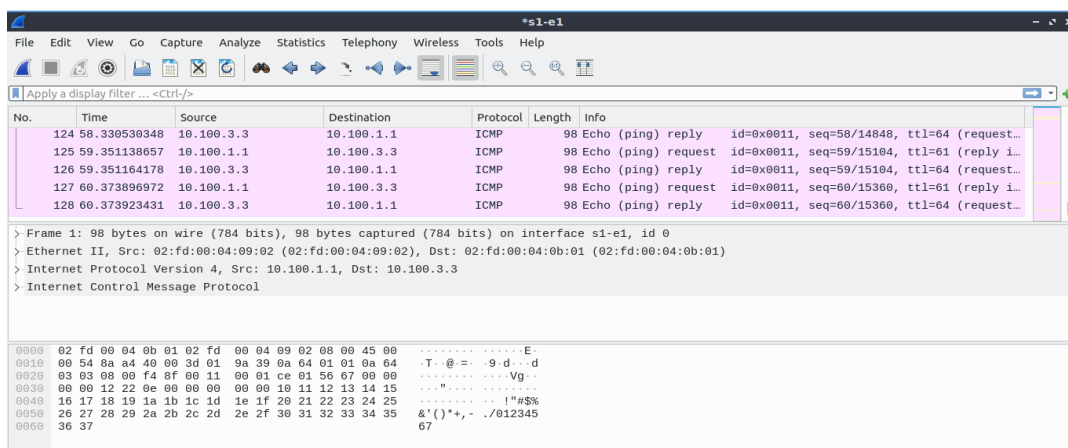
Compruebe el funcionamiento del servicio, siguiendo los siguientes pasos para probar la interconectividad entre h1 y s1:

Desde la consola del host arranque una captura del tráfico en s1 con: wireshark -ki s1-e1 &

Desde h1, pruebe la conectividad con s1 mediante ping 10.100.3.3.

```
h1 - console #1
root@h1:/home/vnx# ping 10.100.3.3
PING 10.100.3.3 (10.100.3.3) 56(84) bytes of data:
64 bytes from 10.100.3.3: icmp_seq=1 ttl=61 time=1.67 ms
64 bytes from 10.100.3.3: icmp_seq=2 ttl=61 time=0.360 ms
64 bytes from 10.100.3.3: icmp_seq=3 ttl=61 time=0.321 ms
64 bytes from 10.100.3.3: icmp_seq=4 ttl=61 time=0.396 ms
64 bytes from 10.100.3.3: icmp_seq=5 ttl=61 time=0.342 ms
64 bytes from 10.100.3.3: icmp_seq=6 ttl=61 time=0.396 ms
64 bytes from 10.100.3.3: icmp_seq=7 ttl=61 time=0.459 ms
64 bytes from 10.100.3.3: icmp_seq=8 ttl=61 time=0.522 ms
64 bytes from 10.100.3.3: icmp_seq=9 ttl=61 time=0.350 ms
64 bytes from 10.100.3.3: icmp_seq=10 ttl=61 time=0.333 ms
64 bytes from 10.100.3.3: icmp_seq=11 ttl=61 time=0.268 ms
64 bytes from 10.100.3.3: icmp_seq=12 ttl=61 time=0.206 ms
```

Después de realizar el ping se muestra el trafico capturado



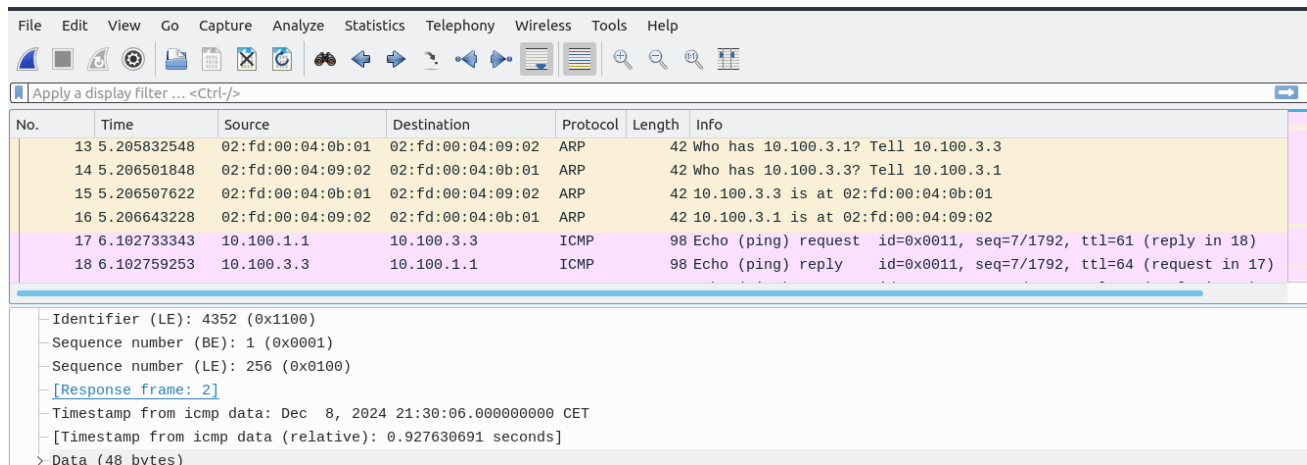
No.	Time	Source	Destination	Protocol	Length	Info
124	58.330530348	10.100.3.3	10.100.1.1	ICMP	98	Echo (ping) reply id=0x0011, seq=58/14848, ttl=64 (request...)
125	59.351138657	10.100.1.1	10.100.3.3	ICMP	98	Echo (ping) request id=0x0011, seq=59/15104, ttl=61 (reply i...
126	59.351164178	10.100.3.3	10.100.1.1	ICMP	98	Echo (ping) reply id=0x0011, seq=59/15104, ttl=64 (request...)
127	60.373896972	10.100.1.1	10.100.3.3	ICMP	98	Echo (ping) request id=0x0011, seq=60/15360, ttl=61 (reply i...
128	60.373923431	10.100.3.3	10.100.1.1	ICMP	98	Echo (ping) reply id=0x0011, seq=60/15360, ttl=64 (request...)

> Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface s1-e1, id 0
> Ethernet II, Src: 02:fd:00:04:09:02 (02:fd:00:04:09:02), Dst: 02:fd:00:04:0b:01 (02:fd:00:04:0b:01)
> Internet Protocol Version 4, Src: 10.100.1.1, Dst: 10.100.3.3
> Internet Control Message Protocol

0000 02 fd 00 04 0b 01 02 fd 00 04 09 02 00 45 00E..
0010 00 54 8a a4 40 00 3d 01 9a 39 8a 64 01 01 0a 64 ..T-@=-.9d...d
0020 03 03 08 00 f4 8f 00 11 00 01 ce 01 56 67 00 00Vg..
0030 00 00 12 22 0e 0e 00 00 00 00 10 11 12 13 14 15
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25!""#%&()
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35 &()*+,-./012345
0060 36 37 67

Se guarda la captura con nombre captura-h1s1, para analizarla y adjuntarla como resultado de la práctica, en la ruta ~/shared/sdedge-ns.

Analice el tráfico capturado y explique las direcciones MAC e IP que se ven en los distintos niveles del tráfico, teniendo en cuenta que el servicio ofrecido utiliza NAT.



No.	Time	Source	Destination	Protocol	Length	Info
13	5.205832548	02:fd:00:04:0b:01	02:fd:00:04:09:02	ARP	42	Who has 10.100.3.1? Tell 10.100.3.3
14	5.206501848	02:fd:00:04:09:02	02:fd:00:04:0b:01	ARP	42	Who has 10.100.3.3? Tell 10.100.3.1
15	5.206507622	02:fd:00:04:0b:01	02:fd:00:04:09:02	ARP	42	10.100.3.3 is at 02:fd:00:04:0b:01
16	5.206643228	02:fd:00:04:09:02	02:fd:00:04:0b:01	ARP	42	10.100.3.1 is at 02:fd:00:04:09:02
17	6.102733343	10.100.1.1	10.100.3.3	ICMP	98	Echo (ping) request id=0x0011, seq=7/1792, ttl=61 (reply in 18)
18	6.102759253	10.100.3.3	10.100.1.1	ICMP	98	Echo (ping) reply id=0x0011, seq=7/1792, ttl=64 (request in 17)

Identifier (LE): 4352 (0x1100)
Sequence number (BE): 1 (0x0001)
Sequence number (LE): 256 (0x0100)
[Response frame: 2]
Timestamp from icmp data: Dec 8, 2024 21:30:06.000000000 CET
[Timestamp from icmp data (relative): 0.927630691 seconds]
> Data (48 bytes)

Como podemos observar, NAT está traduciendo la dirección IP origen, la IP de h1 10.20.1.2 se traduce a la IP publica asignada por el vCPE 10.100.1.1, mientras que la dirección IP destino se conserva, NAT no afecta la IP destino lo que permite que el paquete llegue sin problemas.

También podemos decir que las direcciones MAC de destino y origen no se ven afectadas por la NAT, la misma dirección MAC puede estar asociada a varias IP, sabemos que se está alterando la dirección IP ya que para la MAC 02:fd:00:04:09:02 tenemos dos direcciones IP, una nos la da ARP que dice que la dirección MAC coincide con 10.100.3.1 (router isp1), la otra nos la dan los paquetes, que para la misma MAC nos dice que la dirección IP origen es 10.100.1.1, con este resultado podemos decir que se está realizando una traducción de las direcciones correctamente.

A continuación, desde h1 compruebe el camino seguido por el tráfico a otros sistemas del escenario y a Internet utilizando traceroute:

```
traceroute -In 192.168.255.254
traceroute -In 10.100.3.3
traceroute -In 8.8.8.8
```

Explique los resultados de los distintos traceroute, indicando si se corresponden con lo esperado.

El resultado de los traceroute es el que se muestra en la captura siguiente:

```
root@h1:/home/vnx# traceroute -In 192.168.255.254
traceroute to 192.168.255.254 (192.168.255.254), 30 hops max, 60 byte packets
 1 10.20.1.1 0.079 ms 0.010 ms 0.008 ms
 2 192.168.255.254 0.117 ms 0.027 ms 0.026 ms
root@h1:/home/vnx# traceroute -In 10.100.3.3
traceroute to 10.100.3.3 (10.100.3.3), 30 hops max, 60 byte packets
 1 10.20.1.1 0.096 ms 0.053 ms 0.020 ms
 2 192.168.255.254 1.583 ms 1.569 ms 1.531 ms
 3 10.100.1.254 1.556 ms 1.543 ms 1.539 ms
 4 10.100.3.3 1.568 ms 1.476 ms *
root@h1:/home/vnx# traceroute -In 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1 10.20.1.1 0.096 ms 0.009 ms 0.008 ms
 2 192.168.255.254 1.179 ms 1.179 ms 1.213 ms
 3 10.100.1.254 1.220 ms 1.226 ms 1.181 ms
 4 192.168.122.1 1.224 ms 1.149 ms 1.097 ms
 5 * * *
 6 * * 8.8.8.8 6.125 ms
root@h1:/home/vnx#
```

En el primer traceroute -In 192.168.255.254 el destino se alcanza en dos saltos: El primer salto (10.20.1.1), es el router que conecta a h1 con la red 192.168.255.0/24, y el segundo salto (192.168.255.254), es el destino final, NS:sdedge1 que es la Central de Proximidad 1.

En el segundo traceroute -In 10.100.3.3, hace varios saltos hasta llegar al destino, el primer salto es al router r1 (10.20.1.1), el segundo salto es al NS:sdedge1 (192.168.255.254), el tercer salto va hacia el isp1 (10.100.1.254), el cuarto salto va hacia s1 (10.100.3.3), que sería el destino final.

En el tercer traceroute -In 8.8.8.8, también se realizan varios saltos para alcanzar el destino final, el primero, segundo y tercer salto, son iguales que los del caso anterior, al router r1, después al NS:sdedge1, y después hacia el isp1, hasta llegar al gateway del vcpe. Sin embargo, como ahora nos destinamos hacia una IP externa al sistema, el cuarto salto se realiza hacia el Internet real, por lo que comienza a dar saltos fuera de la red hasta llegar al 8.8.8.8.

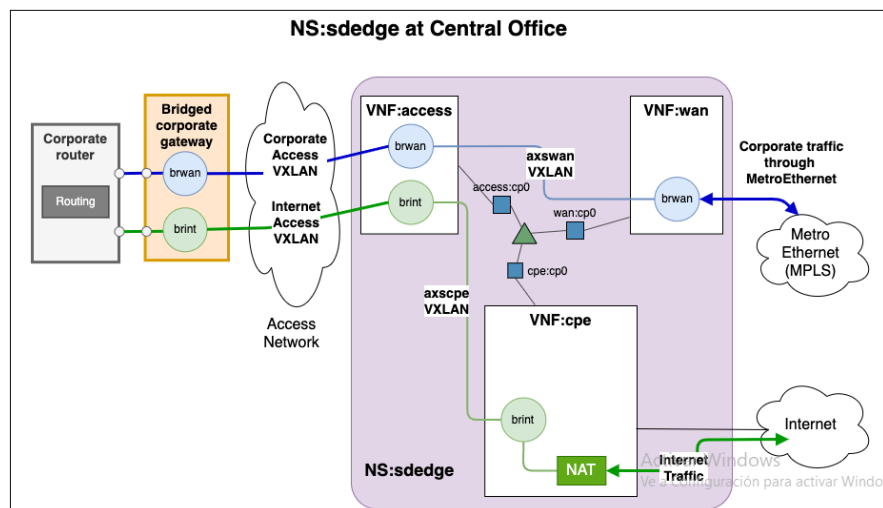
Por los resultados que se observaron anteriormente podemos decir que es el comportamiento esperado, por lo que el sistema funciona correctamente.

Lo siguiente es desinstalar las KNFs mediante uninstall.sh.

```
upm@vnxlab:~/shared/sdedge-ns$ ./uninstall.sh
release "access1" uninstalled
release "cpe1" uninstalled
Error: uninstall: Release not loaded: wan1: release: not found
Error: uninstall: Release not loaded: access2: release: not found
Error: uninstall: Release not loaded: cpe2: release: not found
Error: uninstall: Release not loaded: wan2: release: not found
deployment.apps "access1-accesschart" deleted
deployment.apps "cpe1-cpechart" deleted
upm@vnxlab:~/shared/sdedge-ns$
```

5. Servicio de red sdedge

El servicio de red anterior se va a extender a continuación con una nueva KNF con el objetivo de crear un servicio sdedge, que está preparado para incluir la funcionalidad SD-WAN. La Figura muestra los componentes adicionales de ese servicio.

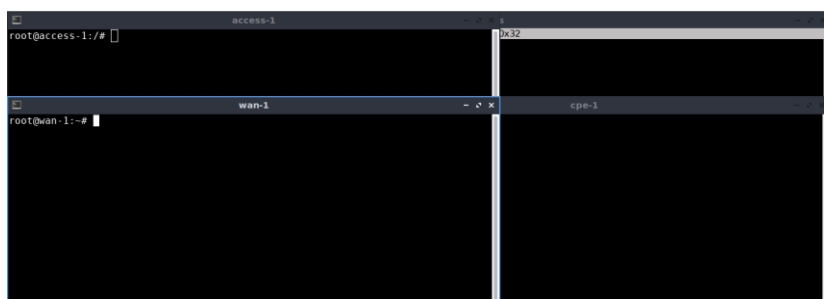


5.1 Instanciación de sdedge1

Cree una instancia del servicio que dará acceso a Internet a la sede 1 y acceso a la red MPLS para la comunicación intra-corporativa, permitiendo conectar con el equipo voip-gw, con ./sdedge1.sh

```
upm@vnxlab: ~/shared/sdedge-ns 15
* Starting ovs-vswitchd
* Enabling remote OVSDDB managers
## 3. En VNF:access agregar un bridge y configurar IPs y rutas
## 4. En VNF:cpe agregar un bridge y configurar IPs y rutas
## 5. En VNF:cpe activar NAT para dar salida a Internet
Adding NAT rules (int_if=brint, ext_if=net1)
Adding rule #1...
MASQUERADE all opt -- in * out net1 0.0.0.0/0 -> 0.0.0.0/0
Adding rule #2...
ACCEPT all opt -- in net1 out brint 0.0.0.0/0 -> 0.0.0.0/0 state RELATED,ESTABLISHED
Adding rule #3...
ACCEPT all opt -- in brint out net1 0.0.0.0/0 -> 0.0.0.0/0
## 1. Obtener IPs de las VNFs
IPACCESS = 10.1.138.9
IPCPE = 10.1.138.16
IPWAN = 10.1.138.17
## 2. Iniciar el Servicio OpenVirtualSwitch en wan VNF
* /etc/openvswitch/conf.db does not exist
* Creating empty database /etc/openvswitch/conf.db
* Starting ovsdb-server
* Configuring Open vSwitch system IDs
* Starting ovs-vswitchd
* Enabling remote OVSDDB managers
## 3. En VNF:access agregar un bridge y sus vxlan
## 4. En VNF:wan agregar el conmutador y su vxlan
--
k8s_sdedge_start.sh
K8s deployments para la red 1:
deploy/access1-accesschart
deploy/cpe1-cpechart
deploy/wan1-wanchart
upm@vnxlab:~/shared/sdedge-ns$
```

Acceder a los terminales de las KNFs usando el comando: bin/sdw-knf-consoles open 1



Realice pruebas de conectividad básicas con ping y traceroute entre las tres KNFs para comprobar que hay conectividad, a través de las direcciones de la interfaz eth0.

La interfaz eth0 de access-1 con una dirección IP 10.1.138.9

La interfaz eth0 de cpe-1 con una dirección IP 10.1.138.16

La interfaz eth0 de wan-1 con una dirección IP 10.1.138.17

Se comprueba la conectividad con los siguientes pings:

Ping desde access-1 hacia cpe-1 y wan-1

Ping desde cpe-1 hacia access-1 y wan-1

Ping desde wan-1 hacia access-1 y cpe-1

```
access-1
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 33 bytes 2096 (2.0 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@access-1:~# ping 10.1.138.16
PING 10.1.138.16 (10.1.138.16): 56 data bytes
64 bytes from 10.1.138.16: icmp_seq=0 ttl=63 time=0.106 ms
64 bytes from 10.1.138.16: icmp_seq=1 ttl=63 time=0.302 ms
64 bytes from 10.1.138.16: icmp_seq=2 ttl=63 time=0.207 ms
64 bytes from 10.1.138.16: icmp_seq=3 ttl=63 time=0.153 ms
64 bytes from 10.1.138.16: icmp_seq=4 ttl=63 time=0.208 ms
^C--- 10.1.138.16 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.106/0.195/0.302/0.065 ms
root@access-1:~# ping 10.1.138.17
PING 10.1.138.17 (10.1.138.17): 56 data bytes
64 bytes from 10.1.138.17: icmp_seq=0 ttl=63 time=0.101 ms
64 bytes from 10.1.138.17: icmp_seq=1 ttl=63 time=0.154 ms
64 bytes from 10.1.138.17: icmp_seq=2 ttl=63 time=0.202 ms
64 bytes from 10.1.138.17: icmp_seq=3 ttl=63 time=0.143 ms
^C--- 10.1.138.17 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.101/0.150/0.202/0.036 ms
root@access-1:~#

cpe-1
root@cpe-1:~# ping 10.1.138.9
PING 10.1.138.9 (10.1.138.9): 56 data bytes
64 bytes from 10.1.138.9: icmp_seq=0 ttl=63 time=0.106 ms
64 bytes from 10.1.138.9: icmp_seq=1 ttl=63 time=0.092 ms
64 bytes from 10.1.138.9: icmp_seq=2 ttl=63 time=0.216 ms
64 bytes from 10.1.138.9: icmp_seq=3 ttl=63 time=0.071 ms
64 bytes from 10.1.138.9: icmp_seq=4 ttl=63 time=0.156 ms
^C--- 10.1.138.9 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.071/0.128/0.216/0.052 ms
root@cpe-1:~# ping 10.1.138.17
PING 10.1.138.17 (10.1.138.17): 56 data bytes
64 bytes from 10.1.138.17: icmp_seq=0 ttl=62 time=0.706 ms
64 bytes from 10.1.138.17: icmp_seq=1 ttl=62 time=0.223 ms
64 bytes from 10.1.138.17: icmp_seq=2 ttl=62 time=0.162 ms
64 bytes from 10.1.138.17: icmp_seq=3 ttl=62 time=0.231 ms
64 bytes from 10.1.138.17: icmp_seq=4 ttl=62 time=0.193 ms
64 bytes from 10.1.138.17: icmp_seq=5 ttl=62 time=0.279 ms
^C--- 10.1.138.17 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.162/0.299/0.706/0.185 ms
root@cpe-1:~#
```

```

wan-1
PING 10.1.138.9 (10.1.138.9): 56 data bytes
64 bytes from 10.1.138.9: icmp_seq=0 ttl=63 time=0.117 ms
64 bytes from 10.1.138.9: icmp_seq=1 ttl=63 time=0.198 ms
64 bytes from 10.1.138.9: icmp_seq=2 ttl=63 time=0.146 ms
64 bytes from 10.1.138.9: icmp_seq=3 ttl=63 time=0.118 ms
64 bytes from 10.1.138.9: icmp_seq=4 ttl=63 time=0.149 ms
^C--- 10.1.138.9 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.117/0.146/0.198/0.029 ms
root@wan-1:~#
root@wan-1:~# ping 10.1.138.16
PING 10.1.138.16 (10.1.138.16): 56 data bytes
64 bytes from 10.1.138.16: icmp_seq=0 ttl=62 time=0.460 ms
64 bytes from 10.1.138.16: icmp_seq=1 ttl=62 time=0.202 ms
64 bytes from 10.1.138.16: icmp_seq=2 ttl=62 time=0.211 ms
64 bytes from 10.1.138.16: icmp_seq=3 ttl=62 time=0.216 ms
64 bytes from 10.1.138.16: icmp_seq=4 ttl=62 time=0.290 ms
64 bytes from 10.1.138.16: icmp_seq=5 ttl=62 time=0.222 ms
64 bytes from 10.1.138.16: icmp_seq=6 ttl=62 time=1.054 ms
64 bytes from 10.1.138.16: icmp_seq=7 ttl=62 time=0.184 ms
^C--- 10.1.138.16 ping statistics ---
8 packets transmitted, 8 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.184/0.355/1.054/0.277 ms
root@wan-1:~#

```

5.2 Análisis de las conexiones de sdedge1 a las redes externas

El fichero `sdedge1`, junto con los ficheros `k8s_sdedge_start.sh` y `start_sdedge.sh`, contiene los comandos necesarios para realizar las configuraciones necesarias del servicio, accediendo a los contenedores mediante `kubectl`, como se explicó anteriormente.

Acceda al contenido de esos tres ficheros. Compare `k8s_corpcpe_start.sh` con `k8s_sdedge_start.sh`. Y explique las diferencias observadas. Acceda también al contenido del fichero `start_sdedge.sh` que se invoca desde `k8s_sdedge_start.sh`.

```

upm@vnxlab:~/shared/sdedge-ns$ diff k8s_corpcpe_start.sh k8s_sdedge_start.sh
4c4
< # SDWNS: namespace in the cluster vim
---
> # SDWNS: cluster namespace in the cluster vim
25c25,26
< for vnf in access cpe
---
>
> for vnf in access cpe wan
36c37
< for vnf in access cpe
---
> for vnf in access cpe wan
46a48
> export VWAN="deploy/wan$NETNUM-wanchart"
48a51
> ./start_sdedge.sh
50a54
> echo "$(basename "$0")"
53a58
> echo $VWAN
upm@vnxlab:~/shared/sdedge-ns$

```

Al ejecutar el comando `diff` entre los archivos `k8s_corpcpe_start.sh` y `k8s_sdedge_start.sh`, se observan las siguientes diferencias:

El `k8s_sdedge_start.sh` maneja un VNF adicional llamado `wan` en el ciclo `for`, además de los VNFs `access` y `cpe`, lo que sugiere que está configurando un entorno SD-WAN, mientras que el `k8s_corpcpe_start.sh` solo maneja los VNFs `access` y `cpe`.

En `k8s_sdedge_start.sh` se exporta la variable `VWAN`, que parece hacer referencia a la configuración del VNF WAN (`export VWAN="deploy/wan$NETNUM-wanchart"`) que nos dice que este script gestiona también la configuración de la red WAN, mientras que este paso no está presente en `k8s_corpcpe_start.sh`.

En `k8s_sdedge_start.sh` se ejecuta el script adicional `start_sdedge.sh`, el mismo realiza configuraciones adicionales como parte de la infraestructura SD-WAN, mientras que `k8s_corpcpe_start.sh` no ejecuta ningún script adicional.

El `k8s_sdedge_start.sh` incluye mensajes `echo` para mostrar el nombre del script en ejecución y el valor de la variable `VWAN` que con esto tenemos más información sobre la ejecución, mientras que el `k8s_corpcpe_start.sh` no tiene estos mensajes.

Con estos resultados podemos plantear que el script `k8s_sdedge_start.sh`, se enfoca en crear una infraestructura SD-WAN más compleja, incluyendo la gestión de un VNF WAN y realizando configuraciones adicionales a través de `start_sdedge.sh`. Mientras que, el script `k8s_corpcpe_start.sh` está se enfoca más en configuraciones de red tradicionales con solo los VNFs access y cpe.

A partir del contenido de los distintos scripts, analice y describa resumidamente los pasos que se están siguiendo para realizar la conexión a las redes externas y la configuración del servicio, comparándolo con el servicio corpcpe.

Para ello analizaremos a continuación los pasos que se siguen en los scripts `k8s_sdedge_start.sh` y `start_sdedge.sh` para configurar la conexión a redes externas y configurar el servicio, comparado con el servicio corpcpe basado en los scripts `k8s_corpcpe_start.sh` y `start_corpcpe.sh`.

Para la Instalación de los VNFs se utilizan los scripts `k8s_sdedge_start.sh` y `k8s_corpcpe_start.sh`. La diferencia es que el `k8s_sdedge_start.sh` incluye un VNF adicional wan para configuraciones de red WAN, mientras que `k8s_corpcpe_start.sh` solo instala access y cpe.

Para la ejecución de los scripts adicionales, el `k8s_sdedge_start.sh` llama a otro script `start_sdedge.sh`, que se encargará de realizar configuraciones adicionales para el funcionamiento del servicio. Este paso no se encuentra en `k8s_corpcpe_start.sh`.

Los dos scripts `k8s_sdedge_start.sh` y `k8s_corpcpe_start.sh`, realizan configuraciones para los VNFs access y cpe, que incluyen la creación de bridges y la configuración de las direcciones IP y rutas. El `k8s_sdedge_start.sh` configura un VNF WAN adicional, donde gestiona un túnel adicional o una red de acceso más compleja, mientras que `k8s_corpcpe_start.sh` solo se ocupa de los VNFs access y cpe.

Ambos scripts configuran las rutas que el tráfico fluya bien entre los VNFs. En el `k8s_sdedge_start.sh`, se hace también la configuración de rutas adicionales para el VNF WAN. En el VNF cpe, se agrega NAT para permitir que los paquetes salgan hacia Internet, llamando al script para configurar el NAT en `start_sdedge.sh`.

Seguidamente se activa el servicio Open Virtual Switch (OVS) en los VNFs access y cpe, que permite la creación de bridges virtuales que faciliten la conexión entre las interfaces y las redes virtualizadas.

Para realizar la comparación entre los servicios corpcpe y sdedge, podemos decir que el servicio corpcpe, es para interconectar de redes locales (access y cpe), proporcionando conectividad entre las redes del cliente y la infraestructura, mientras que el servicio sdedge, es más complejo ya que incorpora un VNF WAN, lo que implica la configuración de redes de área amplia, como una infraestructura SD-WAN.

Compruebe el funcionamiento del servicio, verificando que sigue teniendo acceso a Internet, y que ahora tiene acceso desde h1 y t1 al equipo voip-gw.

```
t1 - console #1
root@t1:/home/vnx#
root@t1:/home/vnx# ping 10.20.1.1
PING 10.20.1.1 (10.20.1.1) 56(84) bytes of data.
64 bytes from 10.20.1.1: icmp_seq=1 ttl=64 time=0.154 ms
64 bytes from 10.20.1.1: icmp_seq=2 ttl=64 time=0.130 ms
64 bytes from 10.20.1.1: icmp_seq=3 ttl=64 time=0.172 ms
64 bytes from 10.20.1.1: icmp_seq=4 ttl=64 time=0.139 ms
^C
--- 10.20.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3053ms
rtt min/avg/max/mdev = 0.130/0.148/0.172/0.015 ms
root@t1:/home/vnx# ping 10.20.0.254
PING 10.20.0.254 (10.20.0.254) 56(84) bytes of data.
64 bytes from 10.20.0.254: icmp_seq=1 ttl=63 time=2.91 ms
64 bytes from 10.20.0.254: icmp_seq=2 ttl=63 time=0.361 ms
64 bytes from 10.20.0.254: icmp_seq=3 ttl=63 time=0.316 ms
64 bytes from 10.20.0.254: icmp_seq=4 ttl=63 time=0.331 ms
^C
--- 10.20.0.254 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.316/0.979/2.909/1.114 ms
root@t1:/home/vnx#

root@t1:/home/vnx# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=10 ttl=251 time=11.1 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=251 time=10.3 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=251 time=9.54 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=251 time=8.79 ms
64 bytes from 8.8.8.8: icmp_seq=14 ttl=251 time=9.18 ms
64 bytes from 8.8.8.8: icmp_seq=15 ttl=251 time=7.39 ms
^C
```

5.3 Instanciación de sdedge2

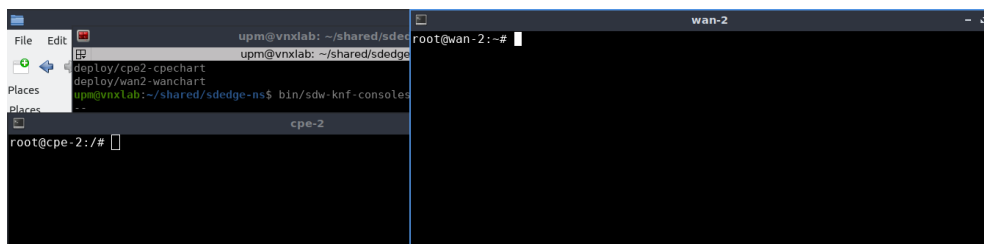
Se va a crear una nueva instancia del servicio sdedge mediante un nuevo fichero sdedge2.sh. Esta instancia permitirá dar acceso a la sede 2 tanto a Internet como a la red MPLS.

Deberá entregar el script sdedge2 como parte del resultado de la práctica.

```
~/shared/sdedge-ns/sdedge2.sh - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

sdedge2.sh
1  #!/bin/bash
2  export SDWNS # needs to be defined in calling shell
3  export SIID="$NSID1" # $NSID1, only for OSM, to be defined in calling shell
4  export SNAME="$BASH_SOURCE"
5
6  export NETNUM=2 # used to select external networks (set to 2 for sdedge2)
7
8  # CUSTUNIP: the ip address for the home side of the tunnel
9  export CUSTUNIP="10.255.0.2"
10
11 # CUSTPREFIX: the customer private prefix
12 export CUSTPREFIX="10.20.2.0/24"
13
14 # VNFTUNIP: the ip address for the vnf side of the tunnel
15 export VNFTUNIP="10.255.0.1"
16
17 # VCPEPUBIP: the public ip address for the vcpe
18 export VCPEPUBIP="10.100.2.1"
19
20 # VCPEGW: the default gateway for the vcpe
21 export VCPEGW="10.100.2.254"
22
23 # HELM SECTION
24 ./k8s_sdedge_start.sh
25
```

Acceda a los terminales de las VNFs



Y realice pruebas de conectividad básicas con ping y traceroute entre las tres KNFs para comprobar que hay conectividad, a través de las direcciones de la interfaz eth0. Observe que las direcciones asignadas son diferentes de las asignadas a las KNFs de sdedge1.

Eth0 de cep-2 es 10.1.138.18, la de wan-2 es 10.1.138.19 y la de Access-2 es 10.1.138.15, lo cual se demuestra que son diferentes a las KNFs de sdedge1. Se comprueba la conectividad entre los terminales.

```
File Edit upm@vnxlab: ~/shared/sdedge upm@vnxlab: ~/shared/sdedge
cpe-2 wan-2
noqueue master ovs-system state UNKNOWN group default
link/ether d2:a8:cf:34:ae:31 brd ff:ff:ff:ff:ff:ff
inet6 fe80::d0a8:cfff:fe34:ae31/64 scope link
valid_lft forever preferred_lft forever
root@cpe-2:~# ping 10.1.138.19
PING 10.1.138.19 (10.1.138.19): 56 data bytes
64 bytes from 10.1.138.19: icmp_seq=0 ttl=62 time=0.116 ms
64 bytes from 10.1.138.19: icmp_seq=1 ttl=62 time=0.117 ms
64 bytes from 10.1.138.19: icmp_seq=2 ttl=62 time=0.117 ms
64 bytes from 10.1.138.19: icmp_seq=3 ttl=62 time=0.117 ms
^C--- 10.1.138.19 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.116/0.406/0.820/0.117 ms
root@cpe-2:~# ping 10.1.138.15
PING 10.1.138.15 (10.1.138.15): 56 data bytes
64 bytes from 10.1.138.15: icmp_seq=0 ttl=63 time=0.121 ms
64 bytes from 10.1.138.15: icmp_seq=1 ttl=63 time=0.121 ms
64 bytes from 10.1.138.15: icmp_seq=2 ttl=63 time=0.121 ms
64 bytes from 10.1.138.15: icmp_seq=3 ttl=63 time=0.121 ms
^C--- 10.1.138.15 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.121/0.189/0.342/0.089 ms
root@cpe-2:~# ping 10.1.138.18
PING 10.1.138.18 (10.1.138.18): 56 data bytes
64 bytes from 10.1.138.18: icmp_seq=0 ttl=63 time=0.116 ms
64 bytes from 10.1.138.18: icmp_seq=1 ttl=63 time=0.071 ms
64 bytes from 10.1.138.18: icmp_seq=2 ttl=63 time=0.212 ms
64 bytes from 10.1.138.18: icmp_seq=3 ttl=63 time=0.108 ms
^C--- 10.1.138.18 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.071/0.127/0.212/0.052 ms
```


Para probar el servicio, compruebe que ahora tiene acceso desde h2 y t2 a Internet y al equipo voip-gw. Además, deberá realizar las siguientes pruebas de interconectividad entre las dos sedes:

Desde la consola de la KNF:wan del servicio sdedge1, lance una captura de tráfico en la interfaz net1, que da salida a la red MplsWan: tcpdump -i net1

Desde h1 lance un ping de 3 paquetes a h2: ping -c 3 10.20.2.2

Como resultado de este apartado, incluya el texto resultado de la captura.

```

wan-1
bytes from 10.1.138.16: icmp_seq=7 ttl=62 time=0.184 ms
--- 10.1.138.16 ping statistics ---
packets transmitted, 8 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.184/0.355/1.054/0.277 ms
ot@wan-1:~# tcpdump -i net1
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on net1, link-type EN10MB (Ethernet), capture size 262144 bytes
:20:12.198823 IP6 fe80::1cdd:f1ff:fe7d:4163 > ff02::2: ICMP6, router solicitation, length 16
:21:34.117816 IP6 fe80::4ce:a7ff:fe57:9cae > ff02::2: ICMP6, router solicitation, length 16
:22:06.886727 IP6 fe80::e06a:d2ff:feb4:5ffd > ff02::2: ICMP6, router solicitation, length 16
:32:01.319492 ARP, Request who-has 10.20.0.2 tell 10.20.0.1, length 28
:32:01.322375 ARP, Reply 10.20.0.2 is-at 02:fd:00:04:05:02 (oui Unknown), length 28
:32:01.323582 IP 10.20.1.2 > 10.20.2.2: ICMP echo request, id 23, seq 1, length 64
:32:01.324770 IP 10.20.2.2 > 10.20.1.2: ICMP echo reply, id 23, seq 1, length 64
:32:02.319633 IP 10.20.1.2 > 10.20.2.2: ICMP echo request, id 23, seq 2, length 64
:32:02.319850 IP 10.20.2.2 > 10.20.1.2: ICMP echo reply, id 23, seq 2, length 64
:32:03.333314 IP 10.20.1.2 > 10.20.2.2: ICMP echo request, id 23, seq 3, length 64

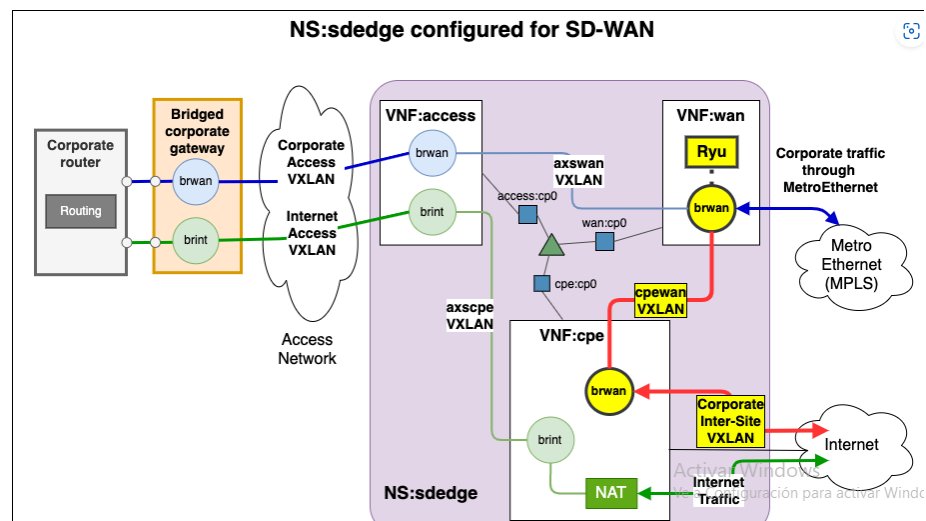
root@h1:~# ping -c 3 10.20.2.2
PING 10.20.2.2 (10.20.2.2) 56(84) bytes of data.
64 bytes from 10.20.2.2: icmp_seq=1 ttl=62 time=7.16 ms
64 bytes from 10.20.2.2: icmp_seq=2 ttl=62 time=0.512 ms
64 bytes from 10.20.2.2: icmp_seq=3 ttl=62 time=0.292 ms

```

El tráfico del ping se muestra en la captura, ya que inicialmente la KNF:wan se configura para conmutar el tráfico entre KNF:access y dicha red.

6. (P) Configuración y aplicación de políticas de la SD-WAN

La Figura muestra resaltados los componentes configurados para el servicio SD-WAN.



Para realizar las configuraciones de SD-WAN sobre el servicio de red sdedge se utiliza el script sdwan1.sh junto a los scripts k8s_sdwan_start.sh y start_sdwan.sh. Acceda al contenido de esos ficheros, así como al contenido de la carpeta json.

A partir de la figura, del contenido de los scripts y de los ficheros json, analice y describa resumidamente los pasos que se están siguiendo para realizar la configuración del servicio y la aplicación de políticas.

Igual que en los apartados anteriores, se irán analizando uno a uno los diferentes scripts involucrados en la configuración del servicio.

```
upm@vnxlab:~/shared/sdedge-ns$ cat sdwan1.sh
#!/bin/bash
export SDWNS # needs to be defined in calling shell
export SIID="$NSID1" # $NSID1, for OSM, to be defined in calling shell

export NETNUM=1 # used to select external networks

export REMOTESITE="10.100.2.1"

# HELM SECTION
```

En el script `sdwan1.sh` configura las diferentes variables de entorno y también el número de la red que se está configurando o la dirección IP de la red externa 2, después llama al script `k8s_sdwan_start.sh`, que configura los despliegues en Kubernetes.

```
./k8s_sdwan_start.shupm@vnxlab:~/shared/sdedge-ns$ cat k8s_sdwan_start.sh
#!/bin/bash

# Requires the following variables
# SDWNS: cluster namespace in the cluster vim
# SIID: id of the service instance
# NETNUM: used to select external networks
# REMOTESITE: the public IP of the remote site vCPE

set -u # to verify variables are defined
: $SDWNS
: $SIID
: $NETNUM
: $REMOTESITE

export KUBECTL="microk8s kubectl"

export VACC="deploy/access$NETNUM-accesschart"
export VCPE="deploy/cpe$NETNUM-cpechart"
export VWAN="deploy/wan$NETNUM-wanchart"

./start_sdwan.sh
```

Analizando el script `k8s_sdwan_start.sh`, se sigue el mismo procedimiento que en los anteriores, configura los despliegues para las VNFs (access, cpe, wan), y llama el script `start_sdwan.sh`, donde se van a configurar rutas, puentes de red, y políticas específicas en las VNFs.

En el script `start_sdwan.sh`, se obtienen las direcciones IP de las VNFs cpe y wan, también se identifica el puerto del servicio WAN utilizando `kubectl` para extraer el puerto del primer nodo. Para la VNF cpe añade una ruta para que cpe se comuniquen con wan a través de un gateway (K8SGW), configura un puente Open vSwitch (brwan) y túneles VXLAN para interconectar cpe con wan y el sitio remoto, también se activan las interfaces y rutas. Para la VNF wan, se activa el controlador SDN (ryu-manager) para gestionar flujos OpenFlow, también se configura el conmutador (brwan) en modo SDN, habilitando OpenFlow con varios protocolos y crea túneles VXLAN para interconectar wan con cpe.

Las políticas que se aplican de SD-WAN están definidas en los archivos JSON y se aplican mediante peticiones `curl` a la API REST del controlador SDN (ryu).

Archivos de políticas JSON:

- from-cpe.json: Configura flujos para tráfico desde cpe.
- to-cpe.json: Configura flujos para tráfico hacia cpe.
- broadcast-from-axs.json: Configura flujos de difusión desde el acceso.
- from-mpls.json: Configura flujos desde MPLS hacia la SD-WAN.
- to-voip-gw.json: Configura flujos hacia el gateway VoIP.

Con esta configuración se consigue mandar el tráfico entre los hosts H1 y H2 por Internet y el tráfico que cursan los teléfonos T1 y T2 por la red MPLS.

```
upm@vnxlab:~/shared/sdedge-ns$ cat start_sdwan.sh
#!/bin/bash

# Requires the following variables
# KUBECTL: kubectl command
# SDWNS: cluster namespace in the cluster vim
# NETNUM: used to select external networks
# VCPE: "pod_id" or "deploy/deployment_id" of the cpd vnf
# VWAN: "pod_id" or "deploy/deployment_id" of the wan vnf
# REMOTESITE: the "public" IP of the remote site

set -u # to verify variables are defined
: $KUBECTL
: $SDWNS
: $NETNUM
: $VCPE
: $VWAN
: $REMOTESITE

if [[ ! $VCPE =~ "-cpechart" ]]; then
    echo ""
    echo "ERROR: incorrect <cpe_deployment_id>: $VCPE"
    exit 1
fi

if [[ ! $VWAN =~ "-wanchart" ]]; then
    echo ""
    echo "ERROR: incorrect <wan_deployment_id>: $VWAN"
    exit 1
fi

CPE_EXEC="$KUBECTL exec -n $SDWNS $VCPE --"
WAN_EXEC="$KUBECTL exec -n $SDWNS $VWAN --"
WAN_SERV="{ $WAN/deploy/ / }"

# Router por defecto inicial en k8s (calico)
K8SGW="169.254.1.1"

## 1. Obtener IPs y puertos de las VNFs
echo "## 1. Obtener IPs y puertos de las VNFs"

IPCPE="$CPE_EXEC hostname -I | awk '{print $1}'"
echo "IPCPE = $IPCPE"

IPWAN="$WAN_EXEC hostname -I | awk '{print $1}'"
echo "IPWAN = $IPWAN"

PORTWAN="$KUBECTL get -n $SDWNS -o jsonpath='{.spec.ports[0].nodePort}'" service $WAN_SERV
echo "PORTWAN = $PORTWAN"

## 2. En VNF:cpe agregar un bridge y sus vxlan
echo "## 2. En VNF:cpe agregar un bridge y configurar IPs y rutas"
$CPE_EXEC ip route add $IPWAN/32 via $K8SGW
$CPE_EXEC ovs-vsctl add-br brwan
$CPE_EXEC ip link add cpewan type vxlan id 5 remote $IPWAN dstport 8741 dev eth0
$CPE_EXEC ovs-vsctl add-port brwan cpewan
$CPE_EXEC ifconfig cpewan up
$CPE_EXEC ip link add srlsr2 type vxlan id 12 remote $REMOTESITE dstport 8742 dev net$NETNUM
$CPE_EXEC ovs-vsctl add-port brwan srlsr2
$CPE_EXEC ifconfig srlsr2 up
```

```
## 3. En VNF:wan arrancar controlador SDN"
echo "## 3. En VNF:wan arrancar controlador SDN"
$SWAN_EXEC /usr/local/bin/ryu-manager --verbose flowmanager/flowmanager.py ryu.app.ofctl_rest 2>&1 | tee ryu.log &
$SWAN_EXEC /usr/local/bin/ryu-manager ryu.app.simple_switch_13 ryu.app.ofctl_rest 2>&1 | tee ryu.log &
$SWAN_EXEC /usr/local/bin/ryu-manager flowmanager/flowmanager.py ryu.app.ofctl_rest 2>&1 | tee ryu.log &

## 4. En VNF:wan activar el modo SDN del conmutador y crear vxlan
echo "## 4. En VNF:wan activar el modo SDN del conmutador y crear vxlan"

$SWAN_EXEC ovs-vsctl set bridge brwan protocols=OpenFlow10,OpenFlow12,OpenFlow13
$SWAN_EXEC ovs-vsctl set-fail-mode brwan secure
$SWAN_EXEC ovs-vsctl set bridge brwan other-config:datapath-id=0000000000000001
$SWAN_EXEC ovs-vsctl set-controller brwan tcp:127.0.0.1:6633

$SWAN_EXEC ip link add cpewan type vxlan id 5 remote $IPCPE dstport 8741 dev eth0
$SWAN_EXEC ovs-vsctl add-port brwan cpewan
$SWAN_EXEC ifconfig cpewan up

## 5. Aplica las reglas de la sdwan con ryu
echo "## 5. Aplica las reglas de la sdwan con ryu"
RYU_ADD_URL="http://localhost:$PORTWAN/stats/flowentry/add"
curl -X POST -d @json/from-cpe.json $RYU_ADD_URL
curl -X POST -d @json/to-cpe.json $RYU_ADD_URL
curl -X POST -d @json/broadcast-from-axs.json $RYU_ADD_URL
curl -X POST -d @json/from-mpls.json $RYU_ADD_URL
curl -X POST -d @json/to-voip-gw.json $RYU_ADD_URL
curl -X POST -d @json/sdedge$NETNUM/to-voip.json $RYU_ADD_URL

echo "---"
echo "sdedge$NETNUM: abrir navegador para ver sus flujos Openflow:"
upm@vnxlab:~/shared/sdedge-ns$
```

```
upm@vnxlab:~/shared/sdedge-ns$ cd json
upm@vnxlab:~/shared/sdedge-ns/json$ ls
broadcast-from-axs.json from-cpe.json from-mpls.json sdedge1 sdedge2 to-cpe.json to-voip-gw.json
upm@vnxlab:~/shared/sdedge-ns/json$
```

Seguidamente se aplican los cambios sobre el servicio de la sede central 1 y la 2: ./sdwan1.sh y ./sdwan2.sh

```
upm@vnxlab: ~/shared/sdedge-ns 150x32
upm@vnxlab:~/shared/sdedge-ns$ ./sdwan1.sh
## 1. Obtener IPs y puertos de las VNFs
IPCPE = 10.1.138.16
IPWAN = 10.1.138.17
PORTWAN = 32063
## 2. En VNF:cpe agregar un bridge y configurar IPs y rutas
## 3. En VNF:wan arrancar controlador SDN
## 4. En VNF:wan activar el modo SDN del conmutador y crear vxlan
loading app flowmanager/flowmanager.py
loading app ryu.app.ofctl_rest
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app None of DPSet
creating context dpset
creating context wsgi
instantiating app flowmanager/flowmanager.py of FlowManager
Created flowmanager
instantiating app ryu.topology.switches of Switches
instantiating app ryu.app.ofctl_rest of RestStatsApi
instantiating app ryu.controller.ofp_handler of OFPHandler
(5363) wsgi starting up on http://0.0.0.0:8080
## 5. Aplica las reglas de la sdwan con ryu
(5363) accepted ('10.0.1.1', 60346)
No such Datapath: 1
10.0.1.1 - - [09/Dec/2024 00:07:39] "POST /stats/flowentry/add HTTP/1.1" 404 122 0.006732
(5363) accepted ('10.0.1.1', 48963)
No such Datapath: 1
10.0.1.1 - - [09/Dec/2024 00:07:39] "POST /stats/flowentry/add HTTP/1.1" 404 122 0.000680
(5363) accepted ('10.0.1.1', 3222)
No such Datapath: 1
10.0.1.1 - - [09/Dec/2024 00:07:39] "POST /stats/flowentry/add HTTP/1.1" 404 122 0.000976
(5363) accepted ('10.0.1.1', 22171)
```

```

upm@vnxlab:~/shared/sdedge-ns$ ./sdwan2.sh
## 1. Obtener IPs y puertos de las VNFs
IPCPE = 10.1.138.18
IPWAN = 10.1.138.19
PORTWAN = 30797
## 2. En VNF:cpe agregar un bridge y configurar IPs y rutas
## 3. En VNF:wan arrancar controlador SDN
## 4. En VNF:wan activar el modo SDN del conmutador y crear vxlan
loading app flowmanager/flowmanager.py
loading app ryu.app.ofctl_rest
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app None of DPSet
creating context dpset
creating context wsgi
instantiating app flowmanager/flowmanager.py of FlowManager
Created flowmanager
instantiating app ryu.topology.switches of Switches
instantiating app ryu.app.ofctl_rest of RestStatsApi
instantiating app ryu.controller.ofp_handler of OFPHandler
(2589) wsgi starting up on http://0.0.0.0:8080
## 5. Aplica las reglas de la sdwan con ryu
(2589) accepted ('10.0.1.1', 42537)
10.0.1.1 - - [09/Dec/2024 00:10:08] "POST /stats/flowentry/add HTTP/1.1" 200 115 0.006898
(2589) accepted ('10.0.1.1', 28071)
10.0.1.1 - - [09/Dec/2024 00:10:08] "POST /stats/flowentry/add HTTP/1.1" 200 115 0.000711
(2589) accepted ('10.0.1.1', 22535)
10.0.1.1 - - [09/Dec/2024 00:10:08] "POST /stats/flowentry/add HTTP/1.1" 200 115 0.000730
(2589) accepted ('10.0.1.1', 41475)
10.0.1.1 - - [09/Dec/2024 00:10:08] "POST /stats/flowentry/add HTTP/1.1" 200 115 0.000661
(2589) accepted ('10.0.1.1', 63659)
10.0.1.1 - - [09/Dec/2024 00:10:08] "POST /stats/flowentry/add HTTP/1.1" 200 115 0.000780

```

A continuación, deberá realizar las siguientes pruebas de interconectividad entre las dos sedes:

Desde la consola del host arranque una captura del tráfico isp1-isp2 con: `wireshark -ki isp1-e2`

Desde h1 lance un ping a h2 (dir. IP 10.20.2.2). El tráfico atraviesa isp1 e isp2, tal y como puede comprobar en la captura de tráfico.

Se guarda la captura con nombre `captura-h1h2`, para analizarla y adjuntarla como resultado de la práctica.

Analice el tráfico capturado y explique las direcciones MAC e IP que se ven en los distintos niveles del tráfico, teniendo en cuenta que se ha encapsulado por un túnel VXLAN. Para que wireshark decodifique correctamente las cabeceras VXLAN, pinche sobre un paquete y use la opción de menú "Analyze->Decode As...", y luego haga doble click en la columna "Current" y seleccione "VXLAN".

No.	Time	Source	Destination	Protocol	Length	Info
29	154.622135296	02:fd:00:04:09:02	02:fd:00:04:0a:02	ARP	42	Who has 10.100.3.2? Tell 10.100.3.1
30	154.623120026	02:fd:00:04:0a:02	02:fd:00:04:09:02	ARP	42	10.100.3.2 is at 02:fd:00:04:0a:02
31	182.270359952	fe80::1cdd:f1ff:fe7... ff02::2		ICMPv6	120	Router Solicitation from 1e:dd:f1:7d:41:63
32	187.390520473	02:fd:00:04:0a:02	02:fd:00:04:09:02	ARP	42	Who has 10.100.3.1? Tell 10.100.3.2
33	187.390534959	02:fd:00:04:09:02	02:fd:00:04:0a:02	ARP	42	10.100.3.1 is at 02:fd:00:04:09:02

> Frame 31: 120 bytes on wire (960 bits), 120 bytes captured (960 bits) on interface isp1-e2, id 0
 > Ethernet II, Src: 02:fd:00:04:0a:02 (02:fd:00:04:0a:02), Dst: 02:fd:00:04:09:02 (02:fd:00:04:09:02)
 > Internet Protocol Version 4, Src: 10.100.2.1, Dst: 10.100.1.1
 > User Datagram Protocol, Src Port: 45090, Dst Port: 8742
 > Virtual eXtensible Local Area Network
 > Ethernet II, Src: 1e:dd:f1:7d:41:63 (1e:dd:f1:7d:41:63), Dst: IPv6mcast_02 (33:33:00:00:00:02)
 > Internet Protocol Version 6, Src: fe80::1cdd:f1ff:fe7d:4163, Dst: ff02::2

0000	02	fd	00	04	09	02	02	fd	00	04	0a	02	08	00	45	00	E..
0010	00	6a	83	d7	00	00	3f	11	df	e2	0a	64	02	01	0a	64	..j....?..d...
0020	01	01	b0	22	22	26	00	56	5a	99	08	00	00	00	00	00	...""&V Z.....
0030	0c	00	33	33	00	00	00	02	1e	dd	f1	7d	41	63	86	dd	..33.....}Ac..
0040	60	00	00	00	00	10	3a	ff	fe	80	00	00	00	00	00	00	~.....}Ac.....
0050	1c	dd	f1	ff	fe	7d	41	63	ff	02	00	00	00	00	00	00	}Ac.....
0060	00	00	00	00	00	00	00	02	85	00	db	b1	00	00	00	00	}Ac.....
0070	01	01	1e	dd	f1	7d	41	63									}Ac

7. Finalización

Al finalizar Para liberar los despliegues realizados en el clúster, utilice: `./uninstall.sh`

8. Conclusiones

Incluya en la entrega un apartado de conclusiones con su valoración de la práctica, incluyendo los posibles problemas que haya encontrado y sus sugerencias.

En esta práctica dado su explicación de los pasos ha sido fluida y se ha aprendido del estudio del tema en detalle.

Durante la realización, en la última parte dio problemas con la conexión entre h_1 y h_2 , pero aun así se realiza la captura de tráfico y se guarda.

De forma general ha sido una práctica interesante donde aporta los conocimientos necesarios a este tema, y da la posibilidad de realizarla en un escenario bien simulado.